REINFORCEMENT LEARNING

Q-LEARNING

Quote by DeepMind

It is a multi-agent problem with multiple players interacting; there is imperfect information due to a partially observed map; it has a large action space involving the selection and control of hundreds of units; it has a large state space that must be observed solely from raw input feature planes; and it has delayed credit assignment requiring long-term strategies over thousands of steps.

Traditional Q-Learning

- Q-learning learns the action-value function *Q(s, a)*: how good to take an action at a particular state
- *Q is called the action-value function*
- In Q-learning, we build a memory table *Q*[*s, a*] to store *Q*-values for all possible combinations of *s* and *a : traditional approach*
- sample an action from the current state.
  - find out the reward *R* (if any) and the new state *s' (*the new board position).
  - From the memory table, determine the next action *a'* to take which has the maximum *Q(s', a')*
- *Q-learning is about creating the cheat sheet Q.*

Traditional Q-Learning

- take a single move *a* and see what reward *R* can get
- This creates a one-step look ahead
- *R + Q(s', a')* becomes the target that we want *Q(s, a)* to be.

$$\text{target} = R(s, a, s') + \gamma \max_{a'} Q_k(s', a')$$

Algorithm:

Start with $Q_0(s, a)$ for all s, a.

Get initial state s

For k = 1, 2, … till convergence

Sample action a, get next state s'

If s' is terminal:

$$\text{target} = R(s, a, s')$$

Sample new initial state s'

else:

$$\text{target} = R(s, a, s') + \gamma \max_{a'} Q_k(s', a')$$

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha) Q_k(s, a) + \alpha \left[\text{target}\right]$$

$$s \leftarrow s'$$

Drawback of Traditional Q-Learning

- However, if the combinations of states and actions are too large, the memory and the computation requirement for *Q* will be too high.
- To address that, we switch to a deep network *Q* (DQN) to approximate *Q(s, a)*.
- The learning algorithm is called **Deep Q-learning**.
- With the new approach, we generalize the approximation of the Q-value function rather than remembering the solutions.

What challenges does RL face comparing to DL?

- In supervised learning, we want the input to be **i.i.d.** (independent and identically distributed), i.e.
- Samples are randomized among batches and therefore each batch has the same (or similar) data distribution.
- Samples are independent of each other in the same batch.
- If not, the model may be overfitted for some class (or groups) of samples at different time and the solution will not be generalized.
- In addition, for the same input, its label does not change over time.
- This kind of stable condition for the input and the output provides the condition for the supervised learning to perform well.

- *In reinforcement learning, both the input and the target change constantly during the process and make training unstable.*
- **Target unstable**: We build a deep network to learn the values of *Q* but its target values are changing as we know things better. As shown below, the target values for *Q* depends on *Q* itself, we are chasing a non-stationary target.
- There is another problem related to the correlations within a trajectory.
  - Within a training iteration, we update model parameters to move *Q(s, a)* closer to the ground truth.
  - These updates will impact other estimations.
  - When we pull up the Q-values in the deep network, the Q-values in the surrounding states will be pulled up also like a net
  - If we update the network with a sequence of actions in the same trajectory, it magnifies the effect beyond our intention.
  - This destabilizes the learning process and sounds like dogs chasing their tails.

Start with $Q_0(s, a)$ for all s, a.

Get initial state s

For k = 1, 2, ... till convergence

Sample action a, get next state s'

If s' is terminal:

$$\text{target} = R(s, a, s')$$

Sample new initial state s'

else:

$$\text{target} = R(s, a, s') + \gamma \max_{a'} Q_k(s', a')$$

$$\theta_{k+1} \leftarrow \theta_k - \alpha \nabla_\theta \mathbb{E}_{s' \sim P(s'|s,a)} \left[(Q_\theta(s, a) - \text{target}(s'))^2 \right] \Big|_{\theta=\theta_k}$$

$$s \leftarrow s'$$

Chasing a nonstationary target!

Updates are correlated within a trajectory!

What is the fundamental challenge in training RL?

- In RL, we often depend on the policy or value functions to sample actions.
- However, this is frequently changing as we know better what to explore.
- As we play out the game, we know better about the ground truth values of states and actions.
- So our target outputs are changing also. Now, we try to learn a mapping *f* for a constantly changing input and output!
- Luckily, both input and output can converge.
- So if we slow down the changes in both input and output enough, we may have a chance to model *f* while allowing it to evolve

Solutions

- **Experience replay**: For instance, we put the last million transitions (or video frames) into a buffer and sample a mini-batch of samples of size 32 from this buffer to train the deep network.
  - This forms an input dataset which is stable enough for training.
  - As we randomly sample from the replay buffer, the data is more independent of each other and closer to i.i.d
- **Target network**: We create two deep networks ϑ- and ϑ.
  - We use the first one to retrieve *Q* values while the second one includes all updates in the training.
  - After say 100,000 updates, we synchronize ϑ- with ϑ.
  - The purpose is to fix the Q-value targets temporarily so we don't have a moving target to chase.
  - In addition, parameter changes do not impact ϑ- immediately and therefore even the input may not be 100% i.i.d., it will not incorrectly magnify its effect as mentioned before

$$L_i(\theta_i) = \mathbb{E}_{s,a,s',r\sim D}\left(\underbrace{r + \gamma \max_{a'} Q(s', a'; \theta_i^-)}_{\text{target}} - Q(s, a; \theta_i)\right)^2$$

Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^{-} = \theta$

For episode $= 1, M$ **do**

  Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

  For $t = 1$,T **do**

    With probability ε select a random action a_t

    otherwise select $a_t = \text{argmax}_a Q(\phi(s_t), a; \theta)$

    Execute action a_t in emulator and observe reward r_t and image x_{t+1}

    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

    Set $y_j = \begin{cases} r_j & \text{if episode terminates at step j+1} \\ r_j + \gamma \max_{a'} \hat{Q}\left(\phi_{j+1}, a'; \theta^{-}\right) & \text{otherwise} \end{cases}$

    Perform a gradient descent step on $\left(y_j - Q\left(\phi_j, a_j; \theta\right)\right)^2$ with respect to the network parameters θ

    Every C steps reset $\hat{Q} = Q$

  End For

End For

Other tricks

- **ϵ-greedy**
- **Architecture**
- **Double DQN**

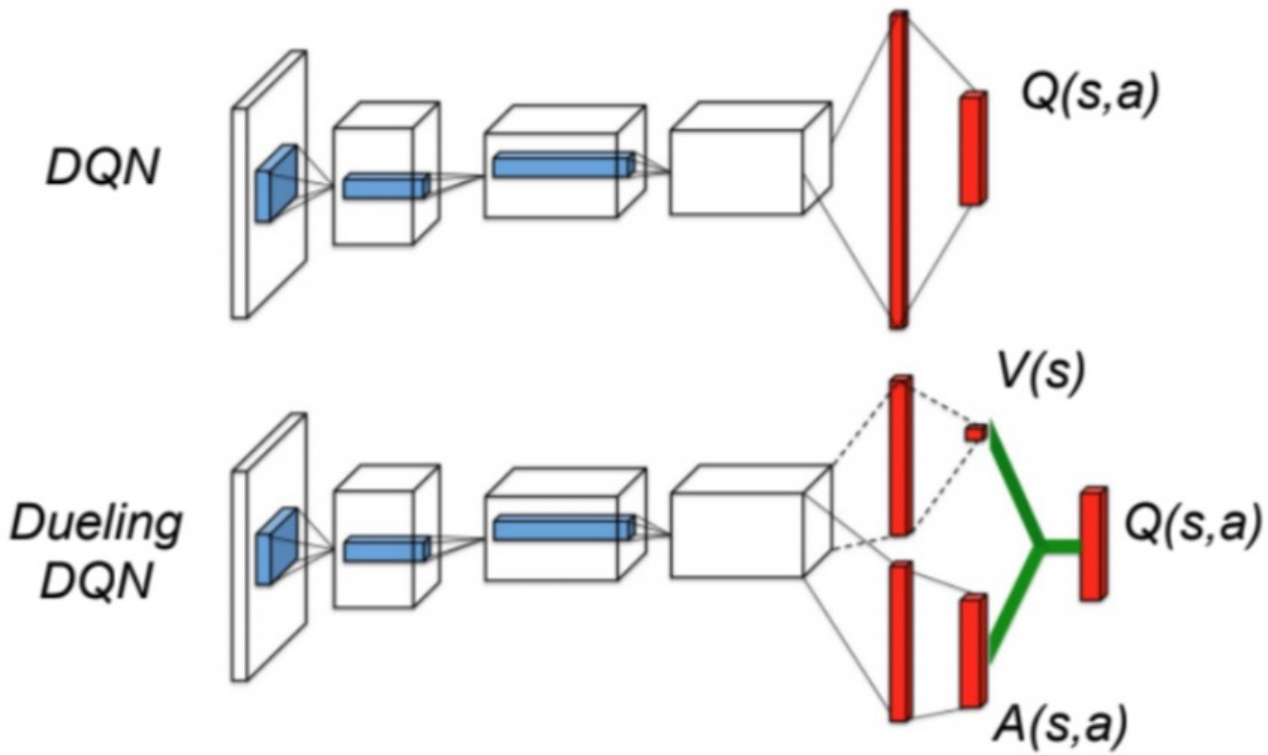
$$L_i(\theta_i) = \mathbb{E}_{s,a,s',r\ D}\left(r + \gamma Q(s', \arg\max_{a'} Q(s', a'; \theta); \theta_i^-) - Q(s, a; \theta_i)\right)^2$$

- **Prioritized Experience Replay :** DQN samples transitions from the replay buffer uniformly.
  - However, we should pay less attention to samples that is already close to the target.
  - We should sample transitions that have a large target gap:

$$\left| r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right|$$

Other tricks contd.

- Therefore, we can select transitions from the buffer
  - based on the error value above (pick transitions with higher error more frequently), or
  - rank them according to the error value and select them by rank (pick the one with higher rank more often).
- Dueling DQN : *Q* is computed with a different formula below with value function *V* and a state-dependent action advantage function *A*
  - Instead of learning *Q*, we use two separate heads to compute *V* and *A*
  - DQN updates the *Q*-value function of a state for a specific action only.
  - Dueling DQN updates *V* which other *Q(s, a')* updates can take advantage of also.


DQN
Q(s,a)
Dueling DQN
V(s)
Q(s,a)
A(s,a)